

Open in app ↗

Sign up

Sign in

Medium

Search

The Glovo Tech Blog · [Follow publication](#)

How to Simulate a Global Delivery Platform

10 min read · Dec 30, 2021



Alex Goldhoorn

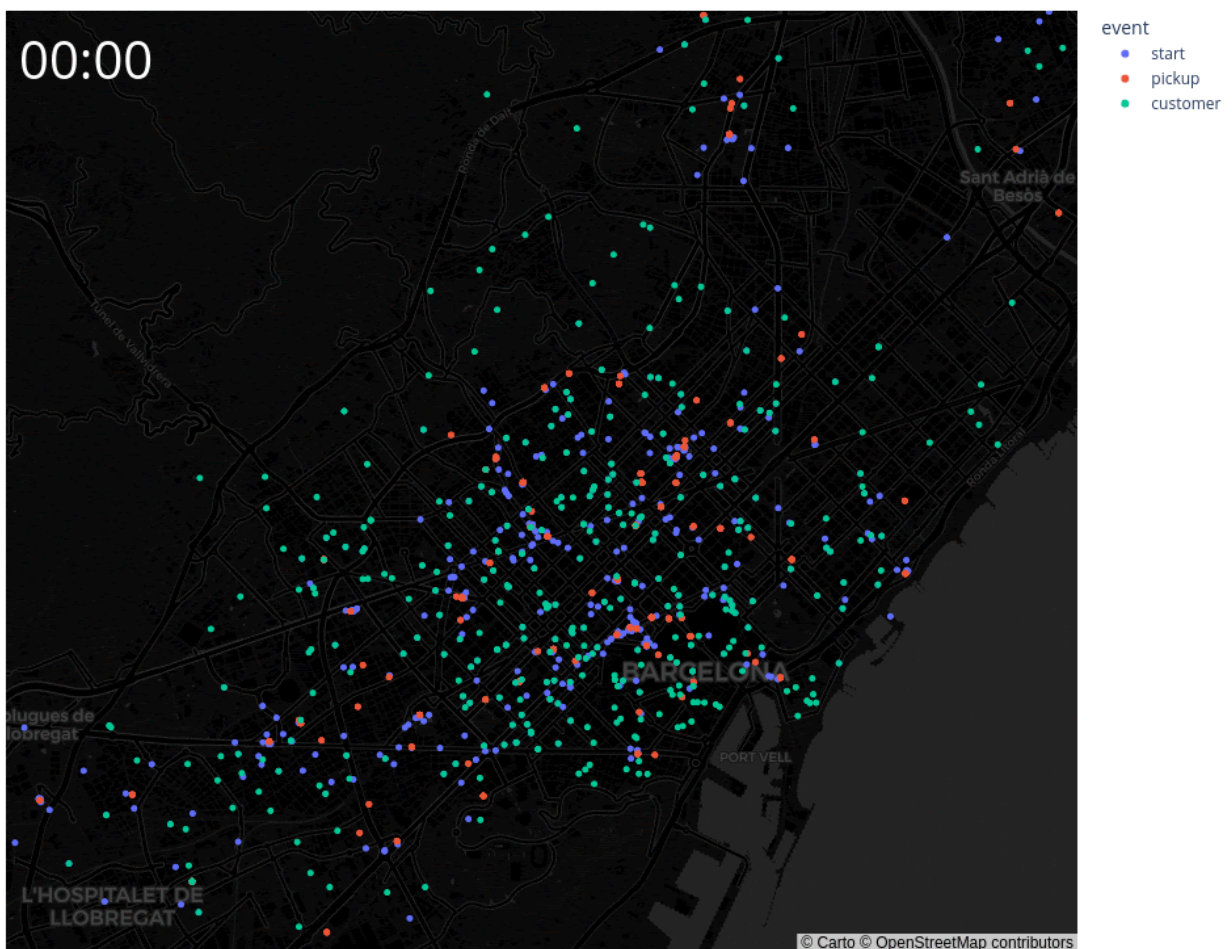
Follow



Listen



Share



One simulated day in Barcelona showing the hourly locations of couriers: start, pickup and delivery (customer).

Glovo is a delivery marketplace that aims at fulfilling very different needs from many different users: 1) customers want to receive their order on time at advantageous costs, 2) couriers want to maximize their earnings while delivering orders that match their possibilities, and 3) partners look for increasing sales by using our digital platform. All of this is possible thanks to a real time optimization performed by the Matching team, where we are responsible for the algorithms that assign every order to its optimal courier. The complexity of this problem is even higher if we consider that Glovo operates in hundreds of cities, with real-time operations of thousands of couriers and a peak-time rate of more than ten orders delivered per second.

Doing improvements and experiments in such a complex operations system is not only costly in time but also has a high business risk. Furthermore, optimizations such as finding the best parameter values for our assignment algorithms are practically impossible to do in the real world or through some simple mathematical equation. Therefore, we have developed and used a Simulator — for over a year now — to test out new features, algorithms, and to find the optimal parameter values for our assignment algorithms.

We simulate the couriers, orders, and the order assignment system (dispatching engine) as different entities. The couriers, like in real life, start at a particular hour at a specific location. Orders are done at a certain time for a specific restaurant. At the same time, the dispatching engine simulates how orders are dispatched to couriers and restaurants. When the courier accepts the order they will go to the pickup location, then wait for preparation and finally do the delivery. The image at the beginning of the article shows how the couriers pass through different states (start, pickup, and arrived at the customer) and locations during a simulated day.

Why Simulate?

If we want to try out new ideas for the operation assignments, we have to verify them. We will show that testing in the real world is much more complex for our case, and simulations have many advantages over these methods.

An idea to test could be to limit the distance for the courier to travel per order to improve delivery times. A/B tests are often used to verify if the feature causes improvements. It does that by activating the new feature for a part of the population and deactivating it for the other part. In the example, we could use the distance limitation for half of the couriers, while not using it for the other half. However, in

practice, this is not a correct comparison because the assignment of a courier to an order influences the orders that can be assigned to a courier from the other group. This kind of problem is called network effect. Other, more complex, experimentation methods can handle network effects, for example, switchback testing or synthetic control.

Simulations, however, have two significant advantages over real-world experimentation. First, the execution of the experiments is much lower: hours or minutes for simulation, whereas real-world experiments typically require several weeks.

Second, simulations are done in a controlled environment, allowing to repeat simulations in similar conditions. Whereas in the real world, external conditions such as weather, special traffic conditions, or side-effects from other experiments can influence the results of the experiments.

Finally, the previous two advantages allow using the simulator to find the best values for parameters we have in the operations system. For example, we can use the simulator to find the best maximum distance for the courier to travel per order. It would be practically infeasible to tune this in the real world since we would require a very long time to try out several parameter values.

How to Simulate?

First, we should decide on what kind of simulations we need, here we will give an overview of the most important types and what type we used and why. A more extensive overview of simulation types can be found in [Simulation and Modelling to Understand Change](#) (by M. Leonelli, 2021).

Stochastic vs. Deterministic

A deterministic simulation would, given the same input, always return the same output. In practice that would mean for us that having the same orders, couriers and configuration (i.e. any used parameters) we should always do the same assignments and therefore end up with the same delivery times.

A stochastic simulation would not necessarily give the same result for different runs of the simulation with the same input. We can create this variance by adding some noise to different parts of the simulation such as the displacement time of a courier and thereby simulating a possible traffic congestion.

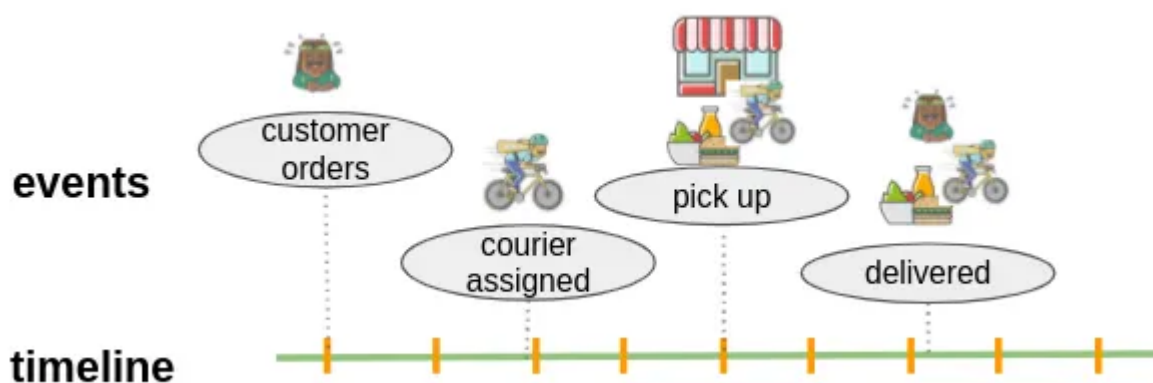
Stochastic simulations in our case have the preference because they allow us to generate slightly different situations with some realistic variations (e.g. traffic delays). Note that here we do greatly depend on the models and therefore should assure that they are as precise as we need them to be. In some cases, deterministic models might be preferred because they are simpler and allow for the comparison of different configurations in the same circumstances.

Continuous vs Discrete Time vs Discrete Event

Next we should decide on the way we simulate time, there are basically three versions (see [Discrete-Event Simulation by P. M. Feldman](#)):

1. **Continuous Time Simulation:** we know the variables in the simulations continuously over time.
2. **Discrete Time Simulation (DTS):** the simulation is done in discrete time steps, i.e. variables are changed every time step.
3. **Discrete Event Simulation (DES):** we only make changes to variables based on certain events, such as for example the arrival of a certain courier to a restaurant to pick up an order.

The first is for when you need to know the exact values at or during a certain time period, for example for physics simulations, which is not our case and we, therefore, focus on the latest two.



Discrete Time Simulation (DTS)

These simulations require us to change the system's state every time step, for example, every simulated minute (see the timeline in the image). In that cycle, we should go through all the entities to see if something has changed. Each of those changes can in fact trigger another change. For example, if the courier arrives at the

restaurant of its assigned order, the system has to check if it has been prepared and if so, the courier should pick it up and should start to move to the delivery location.

Although having this direct time flow in the simulation allows us to clearly define which items have to be checked, at the same time it is costly in the run time and in the amount of code to be written and maintained. Moreover, we would have to define a time step that is small enough to acquire the precision we want, but large enough such that we do not lose too much time in simulating extra steps.

Discrete Event Simulation (DES)

In a Discrete Event Simulation no time steps are simulated, but events, such as an order entity waiting for the assigned courier (entity) to arrive at the pickup location (see the events in the image). And the arrival of the courier to the pickup location event will be triggered after the simulated travel time.

The advantages of this method are that it is easier to design because the events represent the main activities we want to simulate (for example the pickup of an order). Furthermore, we do not need to simulate each time step and therefore the simulations can be faster.

Implementation

There are several [discrete event simulation packages](#), but since we use Python for our main Data Science tasks we decided to go with [SimPy](#), a package that has been maintained for almost 20 years and has an [MIT license](#). In this section, we will shortly describe how to use the package.

Get Alex Goldhoorn's stories in your inbox

Join Medium for free to get updates from this writer.

Enter your email



Subscribe

To run a simulation, we first create an instance of the SimPy Environment:

```
import simpy
env = simpy.Environment()
```

Then we can run one or more processes:

```
env.process(some_process_function())
```

The process function should either wait for some time steps or wait for an event. The following process creates a timeout event that is triggered 5 time-steps in the future (which can represent seconds or minutes for example) and then prints the current environment time (`env.now`) while the simulation is running.

```
def some_process_function():
    while True:
        yield env.timeout(5)
        print(env.now)
```

Finally, we should start and run the simulation, where we can pass until what time step we want to simulate:

```
env.run(until=100)
```

Internally the SimPy library creates a queue of events to run ordered by simulation time. In the next subsections we will show how events can be triggered either explicitly or by waiting a simulated time and how different simulated (parallel) processes can be created.

Timeout

To simulate for example a travel time we use the `Environment.timeout()` function:

```
yield self.env.timeout(travel_time)
...
agent_arrived_event.succeed()
```

Here the process waits for a certain time, simulating the travel time. Once finished, we trigger an event: `agent_arrived_event` .

Events

Events allow us to trigger other processes, for example, the `agent_arrived_event` in the previous example. The event is initialized as:

```
agent_arrived_event = env.event()
```

In a process that needs that event, for example, a restaurant, it will actively wait for that event to continue:

```
yield agent_arrived_event
```

As soon as an event is triggered (using its `succeed()` function), it will yield a value and it can continue the process. Note that events can only be triggered once.

Processes

SimPy runs a process until it yields for some event (timeout or trigger event). The process will continue once the event is triggered (i.e. the function yields a value). An example is the `some_process_function()` function shown at the beginning of this section, a process instance is generated as:

```
process = env.process(some_process_function())
```

A process can also be stopped with the interrupt function: `process.interrupt()`, which will raise a `simpy.Interrupt` exception.

With these building blocks, we can create different agents (run as processes) that interact through events.

Simulator Output

Our simulator logs all events, which allows us to do statistical analysis, and it also shows a more human-readable log:

```
12:44:59: Courier 514148: I have done the check in
12:50:22: Order 85134596 : I am waiting to be assigned
12:51:00: Jarvis: Order 85134596 will be stacked to courier 514148
12:51:00: Order 85134596: I have been stacked to a courier
13:00:00: Courier 514148: I will start to deliver the order 85134596
```

13:00:00: Order 85134596 : I have been dispatched to a courier
13:19:48: Order 85134596: I have been dispatched to a partner
13:19:48: Partner: I have received a notification to start preparing an order
13:19:48: Partner: I have started to prepare the order 85134596
13:19:48: Order 85134596: I have been dispatched to a partner
13:19:48: Order 85134596: I have been dispatched to both partner and courier
13:20:21: Order 85134859 : I am waiting to be assigned
13:21:00: Jarvis: Order 85134859 will be stacked to courier 514148
13:21:00: Order 85134859: I have been stacked to a courier
13:29:48: Courier 514148: I have arrived at the pickup point and I will start to wait
13:29:48: partner finished to prepare the order 85134596
13:29:48: Order 85134596: I have been picked up
13:29:48: Courier 514148: I have picked up the order 85134596 and going to delivery
13:29:48: the courier picked up the order 85134596
13:35:01: Courier 514148: I have arrived at delivery point
13:37:01: Courier 514148: I have delivered the order 85134596
13:37:01: Order 85134596: I have have been delivered
13:38:00: Courier 514148: I will start to deliver the order 85134859
13:38:00: Order 85134859 : I have been dispatched to a courier
...

Note: Jarvis is our dispatching engine.

Simulation Validation

Depending on what the simulator is used for and the available resources and data, there are different ways of validation: from an expert human verifying the result to using measured data to calculate a difference with the simulated results. Since we have access to the latter, we for example look at the delivery time distributions, which in its turn can be split into sub-steps such as going to the restaurant and going to the customer. For more information on the simulator validation see [A practical guide for operational validation of discrete simulation models](#) (Leal, F. et al., 2011).

How do we use it?

In Glovo we have been using the simulator to test new features of different kinds, for example:

- To identify the main drivers of courier service and customer delivery times.
- Understand the behaviour of the overall system when we change certain courier behaviour.

- Measure expected delivery time improvements due to changes of algorithms' parameters.
- Find the best algorithm parameters that improve overall delivery times. In the image below we see an example of the distance limit per courier assignment versus the average delivery times for a day in a city.



The average delivery times per courier distance limit for a day in a city.

Conclusion and future work

We have shown how we have created and how we use a simulator at Glovo to test and improve the matching system. The main advantages are being able to test out new features quickly and without risk, moreover, it provides us with a tool for parameters' optimization.

We also have had interest from other teams and we are sure that the simulator can drive quick and safe testing of new features among many other departments in Glovo. We therefore should tackle some system architecture challenges such as coupling it to existing services in production and making the simulator easy to use by any team in Glovo for example having it as a service.

If you think this kind of project sounds interesting to you as a Data Scientist, Analyst, Operations Researcher, or Software Engineer we really would like you to join Glovo!

Thanks to Manuel Bolivar, Francesco Ricci, Stefan Wold, Emanuele Cardelli and Pablo Giner for their feedback!

Simulation

Data Science

Experimentation

Glovo



Follow

Published in The Glovo Tech Blog

847 followers · Last published Dec 2, 2024

Read about our biggest technical challenges and what it's like to work at Glovo.

<https://engineering.glovoapp.com/>



Follow

Written by Alex Goldhoorn

23 followers · 36 following

PhD in Robotics. Data Scientist @ Matching in Glovo. Simulating...

Responses (2)



Write a response

What are your thoughts?